

Prueba Técnica: Mini-Proyecto de Pipeline de Datos

Primera Parte: GitHub

- 1) Tienes que crear una cuenta de GitHub si no la tienes. Es gratuito, así que no debería de haber problema.
- 2) Crea un repositorio nuevo. Tiene que ser público para que lo podamos revisar (nos tendrás que pasar el enlace al repo).
- 3) A partir de ahora deberás hacer los ejercicios a continuación. Cada apartado numerado debería de ser un único commit (el mensaje que pongas en cada uno es libre, pero como sugerencia puedes poner el nombre del ejercicio y el número del apartado), pero puedes hacer tantos commits como necesites.

Segunda Parte: Python + GCP + BQ

- 1) Tienes que crear un script de Python que se conecte a una API y se descargue los datos (con 100 registros más que suficiente).
Tiene que haber una clase para descargar de la API y otra para subirla a BigQuery. Haz un commit con esta parte y súbelo al repositorio como “parte2” (puedes hacer los commits que quieras pero indicando cada parte)
- 2) Los resultados de la conexión a la API los tienes que cargar en un DATASET que te crees en Bigquery.
 - a) El Dataset que reciba los datos de la API debe seguir esta nomenclatura: `SANDBOX_<nombre de tu aplicación>`

Para esta parte, cuando lo tengas, puedes adjuntar captura de pantalla y subir el fichero que genera la tabla al repositorio.

- 3) Vas a transformar los datos del sandbox. Dentro del repositorio, debe haber una carpeta `sql/` con al menos un archivo:
 1. **transform.sql**: Este archivo debe contener una única consulta SQL que:
 - Lea los datos de la tabla almacenada en `SANDBOX_<nombre de tu aplicación>`
 - Realice alguna transformación simple. Por ejemplo: eliminar posibles duplicados del día, añadir una columna con la fecha en que se ejecuta la transformación...
 - Inserte el resultado transformado en la tabla `INTEGRATION.integration_prueba_tecnica`.
 - **Requisito clave**: La consulta debe ser **idempotente**. Es decir, si se ejecuta varias veces sobre los mismos datos crudos del día, el resultado en la tabla final debe ser el mismo (no debe generar duplicados).

Puntos a tener en cuenta:

1. GCP te va a pedir siempre una tarjeta para poder darte de alta. Forma parte de la verificación. No harán ningún cargo pues es todo libre de coste.
2. Autenticación del Script: El script de Python necesita credenciales para poder interactuar con BigQuery. El método estándar es: Crear una Cuenta de Servicio (Service Account). Asignarle los roles de IAM necesarios (ej. Editor de datos de BigQuery y Usuario de trabajos de BigQuery). Descargar la clave de la cuenta de servicio en formato JSON. Configurar la variable de entorno `GOOGLE_APPLICATION_CREDENTIALS` para que apunte a la ruta de ese archivo JSON.

Tercera Parte: Airflow

Para esta prueba vas a necesitar tener el módulo Airflow instalado en local o en una máquina de docker para poder lanzarlo.

- 1) Define un DAG llamado **test** que se ejecuta cada día a las 3:00 UTC, con los siguientes argumentos por defecto:

```
from datetime import datetime, timedelta
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(1900, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(seconds=5)
}
```

- 2) Incluye las tareas **start** y **end** como **DummyOperator** y que end vaya detrás de start en el DAG.
- 3) Define una lista de tareas dummy **task_n** con **N** tareas, donde cada tarea con **n** par dependa de todas las tareas impares.
- 4) Define un nuevo operador **TimeDiff** que parta del **BaseOperator**, que reciba una fecha (**diff_date**) como entrada y muestre la diferencia con la actual. Crea una tarea nueva con el operador.
- 5) ¿Qué es un Hook? ¿En qué se diferencia de una conexión? Puedes responder en un comentario dentro del código.